

Enterprise MLOps Churn Prediction System

Design and Build a Mini Production ML System

PRODUCTION-ORIENTED PROTOTYPE | VERIFIED 08 AUG 2026

Python 3.9.6

TensorFlow 2.13.0

Keras 2.13.1

scikit-learn 1.3.0

FastAPI 0.103.1

pandas 2.0.3

NumPy 1.24.3

MLflow 2.7.1

pytest 7.4.2

prom-client 0.17.1

Problem definition and intended use

The system predicts whether a telecommunications customer will churn. It supports two decisions: an agent requesting a real-time risk score during a customer interaction, and a marketing team scoring the customer base in batch for retention campaigns. The positive class is Churn = Yes.

DATASET

7,043

customers; 21 raw columns

POSITIVE CLASS

26.5%

moderate imbalance

PRIMARY OBJECTIVE

Recall

with AUC guardrail

Operational decision: customers above the chosen probability threshold enter a retention-review queue; the model does not automatically apply an offer or make an adverse customer decision. Scores should be combined with eligibility rules, contact preferences, intervention cost, and agent judgement. The prototype therefore measures discrimination and recall while keeping a human in the loop.

Success criteria

- Validation ROC AUC ≥ 0.80 and recall ≥ 0.75 ; precision and F1 expose retention-budget trade-offs.
- Online p95 latency below 200 ms with model version returned in every prediction.
- Repeatable ingestion, training, evaluation, serving, drift detection, and retraining-decision paths.
- The work optimizes engineering correctness and reproducibility rather than state-of-the-art accuracy.

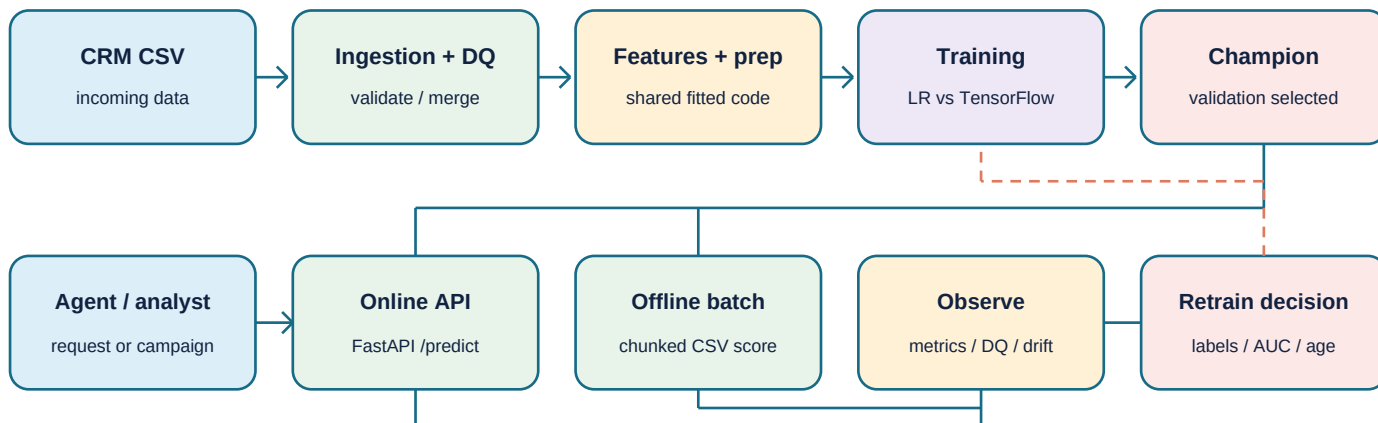
Evidence remains reproducible and artifact-backed.

CODE REPOSITORY

<https://github.com/khshaik/data-science-ml-labs/tree/main/projects/enterprise-mlops-churn-prediction>

Architecture and production workflow

The architecture uses one feature implementation and persisted preprocessing state for training, online requests, and batch scoring. A validation-based champion manifest prevents serving whichever artifact happens to exist most recently. Solid arrows are runtime data paths; the dashed return is an eligible retraining decision that still requires a human or CI job.



Triggers and user/system interactions

Path	Trigger / consumer	Implementation	Current boundary
Online	POST /predict / agent	api.py + shared artifacts	synchronous; verified
Offline	CLI/scheduler / marketing	batch_predict.py	executed; scheduler external
Lifecycle	DQ, labels, AUC, drift, age	drift_detector.py + trigger.py	decision logged; no auto-train

Component and artifact contract

Layer	Code responsibility	Persisted contract
Data + features	ingestion, quality, preprocessing, engineering	threshold + preprocessor
Training	baseline, TensorFlow candidate, train/evaluate	MLflow + eval reports
Serving	FastAPI and batch scoring	current_best.json
Lifecycle	metrics, drift, retraining eligibility	metrics + JSON logs

Dataset and assumptions

The public IBM-style Telco Customer Churn dataset contains customer demographics, services, contract and payment attributes, tenure, monthly charges, total charges, and the Churn label. Customer ID is retained for ingestion deduplication but removed before modelling. Blank TotalCharges values are converted to zero for new customers. The data is a cross-sectional teaching dataset: it demonstrates system design, but it is not a live telecom feed and does not establish causal drivers of churn.

Engineered feature	Construction	Availability / skew control
avg_monthly_charge	TotalCharges / tenure	same deterministic code
service_adoption_score	count of six add-on services	same deterministic code
tenure_category	0-12, 13-24, 25-48, 48+	fixed boundaries
payment_risk_flag	electronic-check indicator	fixed mapping
contract_stability_score	month=1, one-year=2, two-year=3	fixed mapping
high_value_customer	MonthlyCharges > training p75	persisted threshold

Leakage and quality controls

- Raw rows are stratified 60/20/20 before any data-derived feature is fitted. Encoders and scaler fit only on training rows.
- Quality gates cover schema, missing rates, numeric ranges, duplicate IDs, and logical consistency.
- The current batch passed blocking checks. Fifty-nine charge-history deviations are retained as a non-blocking warning because current monthly price need not equal historical average price.
- Batch ingestion merges new CSV data, keeps the latest customer record, and writes a timestamped JSON audit summary.

Offline and online availability is explicit. All six features can be calculated from request-time account fields. The only learned feature parameter is the high-value threshold; it is fitted on training rows, serialized, and loaded for both serving modes. This avoids recomputing a percentile from a single online request or a recent batch.

DEMO EVIDENCE 1/4 - DATA QUALITY RUN

```
rows=7043 columns=21 overall_passed=True
schema=PASS | missing=PASS | ranges=PASS | duplicates=PASS
consistency=WARNING: 59 rows exceed 20% charge-history deviation
artifact: artifacts/logs/data_quality_report_20260808_040152.json
```

Model development and offline evaluation

The baseline is balanced logistic regression. The candidate remains TensorFlow 2.13 - not an sklearn MLPClassifier - with hidden layers [64, 32, 16], seed 42, dropout 0.3, early stopping, learning-rate reduction, and balanced class weights computed from training labels. Validation selects the model; test data is reported separately as the final estimate.

Validation metric	Baseline LR	TensorFlow NN	Winner
ROC AUC	0.8354	0.8300	Baseline
Recall	0.7941	0.7674	Baseline
Precision	0.5174	0.5009	Baseline
F1	0.6266	0.6061	Baseline
Accuracy	0.7488	0.7353	Baseline

BASILINE TEST AUC

0.8429

untouched test set

CANDIDATE TEST AUC

0.8364

untouched test set

CHAMPION

Baseline

lower complexity; better validation

Promotion decision

The candidate passes absolute AUC and recall thresholds, but its validation AUC is 0.0054 lower and all other validation metrics are also lower. It is therefore not promoted: additional latency and model complexity are unjustified without a primary-metric gain.

AUC measures ranking quality across thresholds; recall captures how many true churners reach the intervention queue; precision estimates wasted outreach; and F1 summarizes the recall-precision balance. Accuracy is secondary because the majority class can make it look acceptable even when churners are missed. Final threshold calibration should use observed retention costs and capacity.

DEMO EVIDENCE 2/4 - MODEL COMPARISON

```
VALIDATION baseline_auc=0.8354 candidate_auc=0.8300 delta=-0.0054
VALIDATION baseline_recall=0.7941 candidate_recall=0.7674
GUARDRAILS min_auc=0.80 PASS | min_recall=0.75 PASS | auc_gain FAIL
DECISION KEEP BASELINE -> models/current_best.json
MLFLOW baseline_20260808_041203 FINISHED | candidate_20260808_041229 FINISHED
artifacts/eval/model_comparison.json
```

Request-response contract

FastAPI exposes /predict, /health, /metrics, and optional /explain endpoints. A request supplies the 19 model inputs. The response returns churn probability, Yes/No prediction, Low/Medium/High risk band, champion model version, request latency, and timestamp.

DEMO EVIDENCE 3/4 - LIVE API RESPONSE

```
POST http://127.0.0.1:8000/predict -> HTTP 200
churn_probability: 0.7609
prediction: Yes | risk: High
model_version: baseline_v1.0.0
latency_ms: 8.39 | /metrics verified: true
```

Measured performance

SEQUENTIAL AVG

9.56 ms

100/100 successful

SEQUENTIAL P95

10.21 ms

target < 200 ms

CONCURRENT THROUGHPUT

126.3/s

10 workers; 100/100 success

The benchmark ran on localhost with one application process and a fixed valid request, so it is evidence of functional latency rather than a cloud capacity guarantee. The batch path separately loaded the champion manifest and processed all 7,043 source rows in chunks, retaining customer ID, probability, class, risk band, version, and prediction date.

DEMO EVIDENCE 4/4 - LATENCY BENCHMARK

```
sequential: n=100 success=100 avg=9.56ms p95=10.21ms p99=18.79ms
concurrent: n=100 workers=10 success=100 avg=74.64ms p95=89.01ms
throughput=126.29 requests/sec
batch: 7,043 rows scored successfully; predictions retained as CSV
artifact: artifacts/benchmark_results.json
```

Monitoring and alerts

- Infrastructure: average/p95 latency, throughput, 5xx error rate, service availability, CPU and memory.
- Data: row counts, missing rates, schema changes, data freshness, numeric PSI/KS, and categorical chi-squared tests.
- Model/business: weekly AUC/recall on delayed labels, prediction-rate changes, actual churn, and retention-campaign ROI.
- Executed drift check: 6 features, 0 drifted, drift rate 0.0%.

Retraining and incident response

Retrain when any signal fires: 1,000 new labelled rows, AUC drop above 0.05, drift above 0.30, or 30 days since training. A new candidate must pass validation guardrails before its manifest changes. If CRM changes MonthlyCharges to a currency string, checks reject the batch and preserve the champion. Engineering fixes and replays ingestion; rollback is the unchanged manifest.

Commands, access URLs, and verification boundary

REPRODUCIBLE LOCAL HANDOFF

```
API          uvicorn src.serving.api:app --host 127.0.0.1 --port 8000 -> http://127.0.0.1:8000/docs
MLflow       mlflow ui --backend-store-uri sqlite:///mlflow.db      -> http://127.0.0.1:5000
Metrics      GET http://127.0.0.1:8000/metrics                        -> verified
Offline      python -m src.serving.batch_predict ... | python -m src.monitoring.drift_detector
Lifecycle    python -m src.retraining.trigger | python -m src.training.train --model candidate
Tests        ./venv/bin/python -m pytest tests/unit -q --cov=src -> 96 passed | 0 failed | 69%
Prototype    Streamlit :8501 | Prometheus :9090 | Grafana :3000 (configured, not stack-verified)
```

Alignment, trade-offs, and next work

- Core rubric alignment is complete across problem/data, modelling, inference, production considerations, and documentation. A retained end-to-end ingestion log is the main missing proof artifact.
- Recall is prioritized because missing a churner is assumed costlier than an unnecessary offer; threshold economics require real campaign costs.
- Next: one-hot nominal features, delayed-label AUC and ROI automation, monitoring provisioning, CI repair, and external scheduling.
- Prometheus/Grafana, Docker, CI, SHAP/LIME, and Streamlit remain prototypes rather than evidence of cloud production readiness.

Ownership follows the signal: engineers receive availability, error, latency, schema, and freshness alerts; data scientists own drift, delayed-label evaluation, threshold review, and promotion; business teams receive churn-volume and campaign-outcome summaries. The current 96/96 tests and 69% coverage are verified evidence, not a claim of exhaustive production integration testing. Verification evidence is retained alongside source code.

UNIT TESTS

96/96

all passing

SOURCE COVERAGE

69%

fresh project-local run

SUBMISSION

6 pages

diagram + four evidence panels